

API Documentation

1. Overview

The CampusCare backend is a RESTful API built to support a facility management system for university campus maintenance reporting.

Base URL

http://localhost:3000

API Prefix

/api

Authentication

Most endpoints use **JWT Bearer tokens** issued via Supabase Auth.

Authorization: Bearer <access_token>

2. User Roles

Role	Description
community_member	Students/staff who submit maintenance issues
worker	Maintenance staff who resolve assigned issues
facility_manager	Manages tickets, workers, and system operations
admin	Full system access including user management

3. Common Headers

Header	Value	Required
Authorization	Bearer <token>	For protected routes
Content-Type	application/json	JSON requests
Content-Type	multipart/form-data	File uploads

4. Standard Error Format

```
{ "error": "Error message here" }
```

Common HTTP Errors

- 400 → Bad Request
- 401 → Unauthorized
- 403 → Forbidden
- 404 → Not Found
- 409 → Conflict
- 500 → Internal Server Error

5. Health Check

GET /health

Description: Check if server is running

Auth: None

Response

```
{  
  "status": "UP",  
  "service": "CampusCare API"  
}
```

6. Authentication Routes

6.1 Register User

POST /api/auth/register

Description: Creates a new user account

Auth: None

Request Body

```
{
  "full_name": "Ahmed Hassan",
  "email": "ahmed.hassan@giu-uni.de",
  "password": "SecurePass123",
  "role": "community_member",
  "phone_number": "+201001234567"
}
```

Success Response (201)

```
{
  "user": {
    "id": "uuid",
    "full_name": "Ahmed Hassan",
    "email": "ahmed.hassan@giu-uni.de",
    "role": "community_member"
  },
  "session": {
    "access_token": "jwt_token",
    "refresh_token": "refresh_token"
  }
}
```

6.2 Login

POST /api/auth/login

Description: Authenticates user

Auth: None

Request Body

```
{
  "email": "ahmed.hassan@giu-uni.de",
  "password": "SecurePass123"
}
```

Response (200)

```
{
  "access_token": "jwt_token",
  "refresh_token": "refresh_token",
}
```

```
"user": {  
  "id": "uuid",  
  "full_name": "Ahmed Hassan",  
  "role": "community_member"  
}
```

6.3 Forgot Password

POST /api/auth/forgot-password

Response (200)

```
{  
  "message": "Email verified"  
}
```

6.4 Reset Password

POST /api/auth/reset-password

Request

```
{  
  "email": "user@email.com",  
  "password": "NewPass123"  
}
```

Response

```
{  
  "message": "Password reset successful"  
}
```

6.5 Logout

POST /api/auth/logout

Auth: Any logged-in user

Response

204 No Content

6.6 Get Current User

GET /api/auth/me

Auth: Required

Response

```
{
  "id": "uuid",
  "full_name": "Ahmed Hassan",
  "email": "user@email.com",
  "role": "community_member",
  "status": "active"
}
```

7. Issue Routes

7.1 Create Issue

POST /api/issues

Auth: community_member

Type: multipart/form-data

Fields

Field	Type	Required
title	string	optional
description	string	yes
category	string	yes
building	string	yes

floor	string	yes
room	string	yes
photo	file	optional

Response (201)

```
{
  "id": "uuid",
  "status": "pending",
  "category": "electrical",
  "photo_url": "signed_url",
  "location": {
    "building": "C",
    "floor": "1",
    "room": "C1.04"
  }
}
```

7.2 Get My Issues

GET /api/issues/my

Auth: community_member

Response

```
[
  {
    "id": "uuid",
    "title": "Broken light",
    "status": "assigned",
    "created_at": "timestamp"
  }
]
```

7.3 Get Assigned Issues

GET /api/issues/assigned

Auth: worker

7.4 Get All Issues

GET /api/issues

Auth: facility_manager, admin

7.5 Get Issue by ID

GET /api/issues/:id

Auth: Any authenticated user

7.6 Update Issue Status

PUT /api/issues/:id/status

Auth: facility_manager, worker

Request

```
{  
  "status": "in_progress",  
  "note": "Started work"  
}
```

7.7 Assign Issue

PUT /api/issues/:id/assign

Auth: facility_manager

7.8 Close Issue

PUT /api/issues/:id/close

Auth: facility_manager

7.9 Add Comment

POST /api/issues/:id/comments

Auth: facility_manager, worker

7.10 Upload Completion Photo

POST /api/issues/:id/photo

Auth: worker

Type: multipart/form-data

7.11 Delete Issue

DELETE /api/issues/:id

Auth: facility_manager

8. Manager Routes

8.1 Get Workers

GET /api/manager/workers

Auth: facility_manager, admin

8.2 Update Worker Status

PUT /api/manager/workers/:id/status

9. Admin Routes

9.1 Get All Users

GET /api/admin/users

Auth: admin

9.2 Update User Status

PUT /api/admin/users/:id/status

10. Endpoint Summary

#	Method	Endpoint	Role
1	GET	/health	Public
2	POST	/auth/register	Public
3	POST	/auth/login	Public
4	GET	/issues/my	community_member
5	GET	/issues/assigned	worker
6	GET	/issues	facility_manager/admin

11. Notes

- All timestamps are ISO 8601 format
- All file uploads use Supabase Storage
- All authentication is handled via Supabase JWT
- Role-based access control (RBAC) enforced on backend

Database Schema Documentation

The CampusCare system database is implemented on Supabase (PostgreSQL 15+). The schema consists of five core tables, supported by four custom enumerated types. All tables reside in the public schema with Row Level Security (RLS) enabled to ensure secure and role-based data access.

Enumerated Types

The following custom enumerated types are defined to restrict and standardize values across the database:

- **user_role:** community_member, facility_manager, worker, admin
- **user_status:** active, inactive
- **issue_category:** electrical, plumbing, cleaning, furniture, other
- **issue_status:** pending, assigned, in_progress, resolved, closed

These types ensure data consistency and enforce controlled values for critical system fields.

Table Definitions

Table: users

Stores user profile information and is linked to Supabase authentication (auth.users). This table is used for role-based access control and system authorization.

Column	Data Type	Constraints
id	uuid	PRIMARY KEY, REFERENCES auth.users(id) ON DELETE CASCADE
full_name	varchar(255)	NOT NULL

email	varchar(255)	NOT NULL, UNIQUE
phone_number	varchar(20)	NULL
role	user_role	NOT NULL
status	user_status	NOT NULL, DEFAULT 'active'
created_at	timestampz	NOT NULL, DEFAULT now()

Foreign Keys:

- id → auth.users(id) ON DELETE CASCADE

Indexes:

- users_role_idx (role)
- users_status_idx (status)

Table: locations

Stores physical campus locations where issues are reported. Each record represents a unique location.

Column	Data Type	Constraints
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()

building	varchar(255)	NOT NULL
floor	varchar(50)	NOT NULL
room	varchar(50)	NOT NULL
created_at	timestampz	NOT NULL, DEFAULT now()

Constraints:

- UNIQUE (building, floor, room)

Foreign Keys:

- None

Table: issues

Central table of the system that stores all reported maintenance issues, their status, assignments, and related metadata.

Column	Data Type	Constraints
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()
title	varchar(255)	NOT NULL
description	text	NOT NULL

category	issue_category	NOT NULL
status	issue_status	NOT NULL, DEFAULT 'pending'
photo_url	text	NULL
completion_photo_url	text	NULL
location_id	uuid	NOT NULL, REFERENCES locations(id) ON DELETE RESTRICT
submitted_by	uuid	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
assigned_to	uuid	REFERENCES users(id) ON DELETE SET NULL
created_at	timestampz	NOT NULL, DEFAULT now()
resolved_at	timestampz	NULL
closed_at	timestampz	NULL

Foreign Keys:

- location_id → locations(id) ON DELETE RESTRICT

- submitted_by → users(id) ON DELETE CASCADE
- assigned_to → users(id) ON DELETE SET NULL

Indexes:

- issues_submitted_by_idx (submitted_by)
- issues_assigned_to_idx (assigned_to)
- issues_status_idx (status)
- issues_category_idx (category)
- issues_created_at_idx (created_at DESC)

Table: issue_comments

Stores comments added to issues by authenticated users for communication and updates.

Column	Data Type	Constraints
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()
issue_id	uuid	NOT NULL, REFERENCES issues(id) ON DELETE CASCADE
author_id	uuid	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
body	text	NOT NULL, CHECK (length(trim(body)) > 0)
created_at	timestampz	NOT NULL, DEFAULT now()

Foreign Keys:

- issue_id → issues(id) ON DELETE CASCADE
- author_id → users(id) ON DELETE CASCADE

Indexes:

- issue_comments_issue_idx (issue_id)

Table: issue_status_log

Stores an immutable audit trail of all issue status transitions. Each record captures a change event and preserves system history.

Column	Data Type	Constraints
id	uuid	PRIMARY KEY, DEFAULT gen_random_uuid()
issue_id	uuid	NOT NULL, REFERENCES issues(id) ON DELETE CASCADE
from_status	issue_status	NULL
to_status	issue_status	NOT NULL
changed_by	uuid	NOT NULL, REFERENCES users(id) ON DELETE CASCADE
note	text	NULL
changed_at	timestampz	NOT NULL, DEFAULT now()

Foreign Keys:

- issue_id → issues(id) ON DELETE CASCADE
- changed_by → users(id) ON DELETE CASCADE

Indexes:

- issue_status_log_issue_idx (issue_id, changed_at DESC)

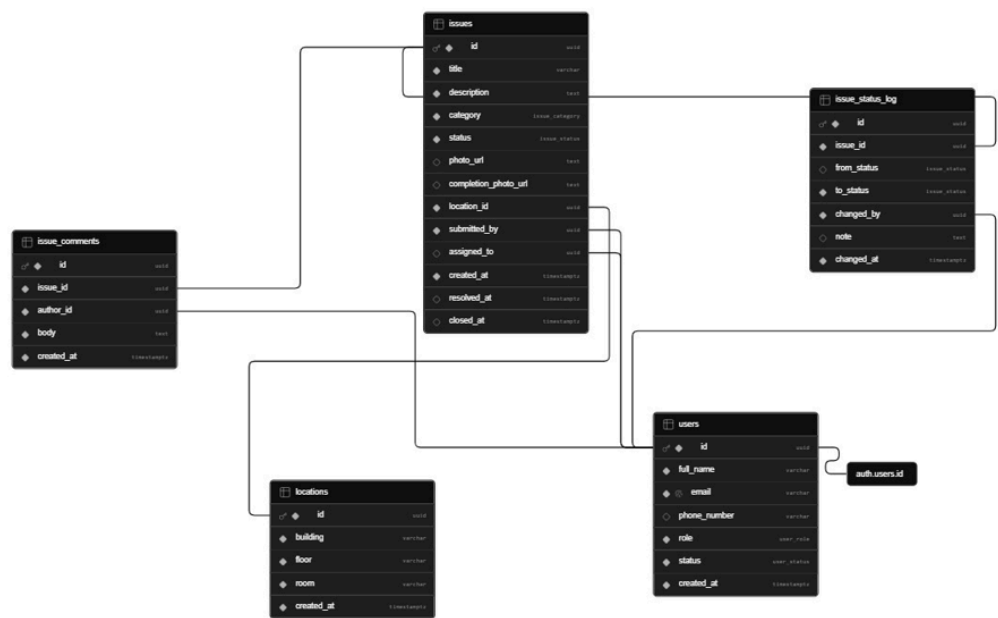
Referential Integrity Summary

The system enforces strict referential integrity to maintain consistency across all relational data:

Foreign Key Column	Source Table	Referenced Table	On Delete Action
id	users	auth.users	CASCADE
location_id	issues	locations	RESTRICT
submitted_by	issues	users	CASCADE
assigned_to	issues	users	SET NULL
issue_id	issue_comments	issues	CASCADE
author_id	issue_comments	users	CASCADE
issue_id	issue_status_log	issues	CASCADE

changed_by	issue_status_log	users	CASCADE
------------	------------------	-------	---------

Entity-Relationship Diagram (ERD):



seed data:

```
insert into public.users (id, full_name, email, phone_number, role, status)
values
('00000000-0000-0000-0000-000000000001',
 'System Admin',      'admin@campuscare.local',  '+2000000000001', 'admin',      'active'),
('00000000-0000-0000-0000-000000000002',
 'Facility Manager',  'manager@campuscare.local', '+2000000000002', 'facility_manager', 'active'),
('00000000-0000-0000-0000-000000000003',
 'Worker One',        'worker@campuscare.local', '+2000000000003', 'worker',      'active'),
('00000000-0000-0000-0000-000000000004',
 'Community Member',  'member@campuscare.local', '+2000000000004', 'community_member', 'active')
on conflict (id) do nothing;

-- ----- Sample locations -----
insert into public.locations (id, building, floor, room) values
('10000000-0000-0000-0000-000000000001', 'Building A', '1', '101'),
('10000000-0000-0000-0000-000000000002', 'Building A', '2', '210'),
('10000000-0000-0000-0000-000000000003', 'Building B', 'Ground', 'Lobby')
on conflict (building, floor, room) do nothing;
```

Users:

id	full_name	email	phone_number	role	status	created_at
00000000-0000-0000-0000-000000000001	System Admin	admin@campuscare.local	+2000000000001	admin	active	2026-05-13 14:56:54.305814+01
00000000-0000-0000-0000-000000000002	Facility Manager	manager@campuscare.local	+2000000000002	facility_manager	active	2026-05-13 14:56:54.305814+01
00000000-0000-0000-0000-000000000003	Worker One	worker@campuscare.local	+2000000000003	worker	active	2026-05-13 14:56:54.305814+01
00000000-0000-0000-0000-000000000004	Community Member	member@campuscare.local	+2000000000004	community_member	active	2026-05-13 14:56:54.305814+01

Locations:

id	building	floor	room	created_at
10000000-0000-0000-0000-000000000001	Building A	1	101	2026-05-13 14:56:54.305814+01
10000000-0000-0000-0000-000000000002	Building A	2	210	2026-05-13 14:56:54.305814+01
10000000-0000-0000-0000-000000000003	Building B	Ground	Lobby	2026-05-13 14:56:54.305814+01

Project Structure Overview

The CampusCare system is organized into two primary components: a React Native mobile application serving as the frontend, and a Node.js/Express REST API serving as the backend. Both components reside within a single repository under the root directory, with a shared documentation folder containing project documentation. This structure ensures clear separation between client-side presentation, server-side logic, and project documentation.

Frontend Structure (Mobile Application)

The mobile application is built using React Native with Expo SDK and follows a modular structure under the `mobile/` directory.

App.js — Application Entry Point

This is the root file of the mobile application. It initializes the navigation system, manages authentication state, and determines the user's role-based access flow. Based on the authenticated user role, it renders the appropriate screens and navigation structure for community members, workers, facility managers, or administrators.

src/screens/ — User Interface Screens

This directory contains all screen components of the application. Each screen represents a full page in the mobile UI and is organized in a flat structure. Screens are role-based, where:

- Community members use screens such as `HomeScreen` and `CreateIssueScreen`
- Workers use `AssignedIssuesScreen`
- Facility managers use `FMDashboardScreen` and `WorkersScreen`
- Administrators use `AdminDashboardScreen`

Shared screens such as `IssueDetailScreen` dynamically adjust their behavior depending on the user role.

src/api/ — Backend Communication Layer

This folder contains all logic responsible for communicating with the backend API. The `client.js` file provides a centralized HTTP request handler that automatically attaches authentication tokens and manages API base configuration. Additional API modules such as `issueApi.js` contain domain-specific functions for handling issue-related requests.

src/auth/AuthContext.js — Authentication State Management

This module manages global authentication state across the application. It handles user login, logout, and session persistence using secure local storage. It also provides access to the current user's identity, role, and authentication status across all screens.

src/components/ — Shared UI Components

This directory contains reusable UI components used across multiple screens. The main component, `IssueCard.js`, provides a standardized visual representation of issue summaries across different dashboards, ensuring UI consistency throughout the application.

src/theme.js — Global Design System

This file defines the global design system of the application, including color palette, spacing, typography, and styling constants. It ensures visual consistency across all screens and components.

Backend Structure (REST API)

The backend is developed using Node.js and Express and follows a structured architecture under the `backend/src/` directory. It is designed using a layered approach that separates routing, business logic, authentication, and configuration.

src/index.js — Server Entry Point

This file initializes the Express application. It sets up global middleware, mounts all API routes under the `/api` prefix, defines a health check endpoint, and starts the server on the configured port.

src/routes/ — API Routing Layer

This directory defines all API endpoints and maps them to their respective controller functions. Each file represents a functional domain:

- `auth.js` handles authentication-related endpoints
- `issues.js` manages issue creation, updates, and lifecycle
- `manager.js` handles facility manager operations

- `admin.js` handles system administration operations

Routes also apply authentication and role-based access control middleware where required.

src/controllers/ — Business Logic Layer

Controllers contain the core business logic of the application. They process incoming requests, interact with the database, and return structured responses.

- `authController.js` manages user authentication flows such as registration and login
- `issueController.js` handles issue lifecycle operations including creation, assignment, updates, and comments
- `UserController.js` manages user and worker-related operations

src/middleware/ — Security and Access Control

Middleware functions handle authentication and authorization.

- `requireAuth.js` validates JWT tokens and retrieves the authenticated user
- `rbacMiddleware.js` enforces role-based access control by restricting endpoints based on user roles

src/config/supabase.js — External Service Configuration

This file initializes Supabase clients used by the backend. It separates:

- A public client for authentication operations
- A service-role client for secure database and storage access

This ensures proper security boundaries between public and administrative operations.

migrations/ — Database Schema Definitions

This folder contains SQL migration files that define and initialize the database schema:

- `001_init.sql` creates tables, relationships, and constraints
- `002_seed.sql` provides optional test and sample data for development

Overall Architecture

The CampusCare system follows a client–server architecture where the mobile frontend communicates exclusively with the backend through a RESTful API. The frontend is responsible for user interaction and presentation, while the backend handles business logic, data processing, and system security.

Authentication is implemented using JSON Web Tokens (JWT) issued by Supabase Auth. After login, the token is stored securely on the client device and included in all subsequent API requests. The backend validates these tokens using authentication middleware.

Role-Based Access Control (RBAC) is enforced at the API level, ensuring that each user role—community member, worker, facility manager, and admin—has access only to permitted actions. This guarantees proper separation of privileges across the system.

The system maintains a strict separation of concerns:

- Frontend → presentation and user interaction
- Backend → business logic and API processing
- Supabase → authentication, database storage, and file handling

This architecture ensures scalability, security, and maintainability of the CampusCare application.